

1. 总体处理链路

文档入库的目标，是把外部文件、目录、网页、代码仓库或云文档转换为 OpenViking 内部的知识节点。入库完成后，系统不仅保存原文或转换后的 L2 内容，还会生成用于检索的 .abstract.md、.overview.md，并把这些信息写入向量索引。

入口

HTTP POST /api/v1/resources
或 SDK/Local Client add_resource()

同步阶段

1. 资源参数校验和身份上下文解析
2. ResourceService 派生目标 URI、注册知识文档回调
3. ResourceProcessor 调用 UnifiedResourceProcessor
4. ParserRegistry 选择解析器，Parser 写入临时 VikingFS
5. TreeBuilder 解析最终 root_uri 和 temp_uri
6. ResourceProcessor 将 temp 文档树移动到正式 AGFS
7. Summarizer 投递 SemanticMsg

异步阶段

8. SemanticProcessor / SemanticDag 自底向上生成 L0/L1
9. Embedding / Vectorize 写入向量索引
10. KnowledgeDocumentRegistry 更新 ready 或 failed 状态

阶段	同步/异步	主要输入	主要输出
接入校验	同步	path/temp_file_id、to、parent、folder_path、headers	合法 source、RequestContext、处理参数
解析切分	同步	source、instruction、strict/include/exclude	ParseResult、temp_dir_path、source_format、warnings
URI 落位	同步	temp_dir_path、scope、to/parent、source_path	root_uri、temp_uri、candidate_uri
正式落盘	同步	temp_uri、root_uri、锁	正式 VikingFS/AGFS 文档树
任务投递	同步	root_uri、temp_uri、document_id、ctx	SemanticMsg 入队，processing_enqueued=true
摘要索引	异步	SemanticMsg、目录树、原文内容	.abstract.md、.overview.md、向量记录
状态完成	异步/同步等待	队列状态、embedding 结果	knowledge document ready/failed、queue_status

阅读方式

后续章节按真实执行顺序展开。每一步都说明入口、执行模块、输入、输出、写入位置和异常处理，读者可以沿着 root_uri 和 document_id 跟完整条链路。